

Manual Técnico

Iter Store — Instalação, operação e API GraphQL

VERSÃO 1.0 · OUTUBRO DE 2026



Sistema projetado e desenvolvido por

Esmerio Neto — EGSN

esmerioneto@gmail.com · © 2026 EGSN. Todos os direitos reservados.

Sumário

1. Visão geral
    1. Arquitetura
    2. Módulos
2. Instalação e ambiente
    1. Requisitos
    2. Passo a passo
    3. Arquivos de configuração
    4. Produção
3. Operação
    1. Tarefas agendadas (cron)
    2. Webhook do Mercado Pago
    3. Logs
    4. Testes
    5. Backup
4. A API GraphQL
    1. Endpoint e formato
    2. Autenticação
    3. Cabeçalhos e CORS
    4. Erros
    5. Limites
    6. Envio de arquivos
5. Exemplos: loja e catálogo
6. Exemplos: compra do visitante
7. Exemplos: conta do cliente
8. Exemplos: integração do admin
    1. Login
    2. Pedidos
    3. Produtos e estoque
    4. Relatórios
9. Códigos de erro
10. Referência das consultas e mutações

Exemplos reais

Todas as requisições deste manual foram executadas contra a API desta instalação em 20/10/2026, e as respostas são as que ela devolveu (dentro de uma transação desfeita depois: nada ficou gravado). Listas longas foram cortadas ("… mais N") e os tokens, abreviados.

1. Visão geral

1.1 Arquitetura

O Iter Store é uma aplicação PHP sem framework externo, com PostgreSQL. Tem duas portas de entrada:

- **Painel administrativo** (`/admin`): telas renderizadas no servidor, sessão por cookie, proteção CSRF (`form_key`) e permissões por perfil (ACL).
- **API GraphQL** (`/core/graphql/index`): um único endpoint para o app, o site e integrações (ERP, atendimento, contabilidade). O contrato completo fica em `schema.graphql`, na raiz do projeto.

Os dados flexíveis (atributos de produto, categoria, cliente e endereço) usam o modelo **EAV**: os campos são criados no painel (Catálogo → Atributos) e aparecem na API em `attributes`, sem mudar o código. As views `customer_flat` e `customer_address_flat` entregam o cliente "achatado" para consultas.

O código fica em módulos (`src/vendor/Iter/<Módulo>`), cada um com `etc/config.json` (nome e dependências), `etc/acl.json` (permissões), `etc/menu.json`, `etc/router.json`, `Setup/` (migrações), `GraphQl/` (a parte da API) e `view/`. Módulos próprios do cliente ficam em `src/app/code/`. Desligar um módulo em Lojas → Configurações → Módulos tira as telas e a API dele, sem apagar os dados.

1.2 Módulos

Módulo	Depende de	O que faz
Core	—	Banco (Adapter, Select), configuração, criptografia dos segredos, log, sessão, limite por IP, migrações e o endpoint GraphQL.
Router	—	Rotas: `/admin/<controller>/<ação>` e as rotas dos módulos (`etc/router.json`).
Admin	—	Login do painel, perfis e permissões (ACL), menu, token da API do admin.
Config	—	Configurações por escopo (padrão / website) e a tela de módulos.
Mail	—	Fila de e-mails, modelos editáveis e e-mails enviados.
Store	—	Websites, grupos e lojas.
Eav	—	Atributos, conjuntos e valores (produto, categoria, cliente, endereço).
Catalog	Eav	Produtos (simples e com grade), categorias, imagens, custo, ordem do app e importação de custos.

Inventory	Catalog	Origens de estoque, quantidade por origem, reservas e o vendável.
Customer	Eav, Store	Clientes PF/PJ, endereços, login do cliente na loja e token da API.
Sales	Catalog, Inventory, Customer, Store	Carrinho, pedido, situações, envio, cancelamento e painel de vendas.
Adminsales	Sales	Vendas → Novo pedido (pedido feito pelo atendimento) e a API correspondente.
Payment	Sales	Formas de pagamento manuais (Pix com chave, pagamento na entrega).
Mercadopago	Payment	Pix, crédito e débito pelo Mercado Pago, webhook, sincronização e estorno.
Shipping	Sales	Frete por faixas, retirada na loja e o registro de provedores.
Melhorenvio	Shipping	Cotação pelo Melhor Envio (OAuth).
Correios	Shipping	Cotação pela API dos Correios (contrato). *Desligado nesta instalação.*
Promotion	Sales, Catalog, Store	Promoções automáticas e cupons.
Message	Customer, Store	Mensagens entre a loja e o cliente.
Log	Admin, Config	Log de operações (auditoria) com antes e depois.
Report	Admin, Config, Sales	Os 26 relatórios, exportação, envio por e-mail e a API de relatórios.

Core, Router, Admin, Config e Mail são obrigatórios. Os outros são ligados em `src/etc/config.json` (pela tela de módulos); o painel recusa desligar um módulo do qual outro ligado depende.

2. Instalação e ambiente

2.1 Requisitos

- PHP 8.1 ou superior com `pdo_pgsql`, `mbstring`, `curl`, `gd`, `zip` e `openssl`.
- Composer. Bibliotecas: webonyx/graphql-php (API), symfony/mailer (e-mail), guzzlehttp/guzzle (Mercado Pago, Melhor Envio, Correios), openspout/openspout (Excel) e dompdf/dompdf (PDF dos relatórios).
- PostgreSQL 15 (há um `docker-compose.yml` pronto para desenvolvimento, ouvindo só em 127.0.0.1:5432; em produção, PostgreSQL instalado no servidor ou gerenciado).
- Para o backup: `restic` e o `pg_dump` / `psql` da mesma versão do PostgreSQL ou mais nova (`postgresql-client-15`).

2.2 Passo a passo

```
composer install
docker compose up -d                                  # PostgreSQL
cp src/etc/database.ini.sample src/etc/database.ini   # credenciais do banco
cp src/etc/mail.ini.sample src/etc/mail.ini           # SMTP
php bin/migrate.php up                                # cria e atualiza as tabelas
php bin/create-admin.php admin 'SenhaForte123' admin@loja.com.br
php -S localhost:8080 -t public                       # só para desenvolvimento
```

O painel abre em `http://localhost:8080/admin` e a API em `http://localhost:8080/core/graphql/index`. Depois de ligar um módulo pela primeira vez, rode `php bin/migrate.php up` de novo. `php bin/migrate.php down` desfaz a última migração.

2.3 Arquivos de configuração

Arquivo	Conteúdo
`src/etc/database.ini`	Host, porta, banco, usuário e senha do PostgreSQL. Fora do Git.
`src/etc/mail.ini`	Transporte de e-mail (SMTP ou `log`, que grava os e-mails em `var/mail`). Fora do Git.
`src/etc/env.ini`	A chave que criptografa os segredos gravados no banco (Access Token do Mercado Pago, senha dos Correios…). Criado no primeiro uso. Entra no backup e no kit de recuperação: sem ele, esses segredos (e a senha do backup) não podem ser lidos.
`src/etc/config.json`	Os módulos ligados. Alterado pela tela de módulos.
`schema.graphql`	O contrato da API. Regere com `php bin/schema-dump.php` depois de mudar a API (o teste do admin falha se ele estiver desatualizado).

As demais configurações (prazos, limites, credenciais de pagamento e frete, CORS, limites da API) ficam no banco (`core_config_data`) e são editadas em Lojas → Configurações, por padrão ou por website.

2.4 Produção

- Raiz web apontando para `public/`; tudo o mais fica fora do alcance do navegador. Todas as requisições vão para `public/index.php` (no Nginx: `try_files $uri /index.php?$query_string;`).
- HTTPS obrigatório: o token da API vai no cabeçalho de cada requisição. Com o certificado instalado, ligue Configurações → Geral → *Usar só HTTPS*: páginas em http são redirecionadas, a API em http é recusada (403), o navegador recebe HSTS e o cookie do painel passa a ser `Secure`.
- Atrás de um balanceador, proxy ou CDN, informe os IPs dele em Configurações → Geral → *Proxies confiáveis*. Sem isso, todos os clientes parecem vir do IP do proxy e o limite por IP bloqueia todo mundo junto; com isso, o IP do cliente e o HTTPS vêm de `X-Forwarded-For` e `X-Forwarded-Proto` (só quando enviados pelo proxy).
- Crie uma chave em Sistema → Chaves da API para cada app e site, configure-os para enviá-la e ligue *Exigir chave do aplicativo*.
- `display_errors` desligado no PHP; os erros vão para `var/log`.
- `var/` (logs, cache, sessões, e-mails) e `public/media/` (imagens enviadas) graváveis pelo usuário do PHP; `public/media/` entra no backup junto com o banco e o `src/etc` (3.5).
- Backup: o cron do `bin/backup.php` ativo, um destino externo em Configurações → Backup e o kit de recuperação guardado fora do servidor (3.5).
- Em CORS, informe só os sites que podem chamar a API pelo navegador.
- `src/etc/database.ini` e `mail.ini` com permissão 600 (só o usuário do PHP lê), como o `env.ini`.
- O painel já envia `X-Frame-Options: DENY`, CSP `frame-ancestors 'none'`, `nosniff` e `Referrer-Policy: same-origin`. A sessão do painel termina depois de 8 h sem uso e 12 h desde o login (Configurações → Segurança do admin), e quando a senha muda. Sair é um POST com `form_key` (`/admin/auth/logoutpost`).

3. Operação

3.1 Tarefas agendadas (cron)

Quando	Script	Para quê
`* * * * *`	`bin/mail-worker.php`	Envia a fila de e-mails. Sem ele, nenhum e-mail sai.
`* * * * *`	`bin/mercadopago-sync-loop.sh`	Roda o mercadopago-sync.php a cada 5 segundos dentro do minuto (com flock), para o Pix ser confirmado rápido. Alternativa mais leve: mercadopago-sync.php a cada 10 minutos.
`15 * * * *`	`bin/order-clean.php`	Cancela os pedidos não pagos depois do prazo de cada forma de pagamento.
`10 0 * * *`	`bin/log-clean.php`	Apaga os arquivos de log e o log de operações mais antigos que o prazo configurado.
`20 0 * * *`	`bin/quote-clean.php`	Apaga carrinhos de visitante e pedidos em montagem do admin parados.
`30 7 * * *`	`bin/report-mail.php`	Envio automático dos relatórios (segunda-feira ou dia 1º, se ligado).
`* * * * *`	`bin/backup.php`	Backup com restic no intervalo de Configurações → Backup (ou ao "Fazer backup agora"); limpeza diária das cópias velhas e teste de restauração mensal. Ver 3.5.

```
* * * * *   php /srv/iterstore/bin/mail-worker.php
* * * * *   /srv/iterstore/bin/mercadopago-sync-loop.sh
15 * * * *  php /srv/iterstore/bin/order-clean.php
10 0 * * *  php /srv/iterstore/bin/log-clean.php
20 0 * * *  php /srv/iterstore/bin/quote-clean.php
30 7 * * *  php /srv/iterstore/bin/report-mail.php
* * * * *   php /srv/iterstore/bin/backup.php >> /srv/iterstore/var/log/backup.log 2>&1
```

Outros scripts: `bin/create-admin.php` (primeiro admin, ou recuperar o acesso), `bin/schema-dump.php` (regera o `schema.graphql`), `bin/migrate.php` e `bin/db-bench.php` (tempo das listagens EAV).

3.2 Webhook do Mercado Pago

A única rota fora do GraphQL e do painel: `POST https://sua-loja/mercadopago/webhook`. Cadastre-a no painel do Mercado Pago (Suas integrações → Webhooks, tópico *Order*) e copie a "assinatura secreta" para Configurações → Pagamento. Com a assinatura salva, avisos com `x-signature` inválida recebem 401, e também os assinados há mais de 10 minutos (o `ts` da assinatura, em segundos ou milissegundos): um aviso capturado e reenviado depois é recusado.

O corpo do aviso só diz qual cobrança mudou: o sistema relê a cobrança na API do Mercado Pago, e o cron `mercadopago-sync` cobre os avisos perdidos.

3.3 Logs

- `var/log/iter-AAAA-MM-DD.log`: erros e avisos do sistema, um arquivo por dia (inclui a causa real de um `INTERNAL_ERROR` da API). Consulta também em Sistema → Log do sistema.
- Log de operações (tabela `log_operation`): quem fez o quê no painel ou pela API, com antes e depois, logins e exportações de relatórios de clientes e financeiros.
- E-mails: tabela `mail_queue`, consultada em Sistema → E-mails enviados.

3.4 Testes

```
composer test
```

Roda `view-selftest`, `db-selftest`, `admin-selftest` (sobe um `php -S` e testa o painel e a API de ponta a ponta), `promotion-selftest`, `report-selftest`, `backup-selftest` (um backup de verdade num repositório descartável, com restauração e zip) e `shipping-provider-selftest` (provedores de frete sem rede). Usam o banco real: criam registros `selftest_*` ou trabalham numa transação desfeita, e limpam tudo ao terminar. Mantêm ligados só os módulos que estavam ligados antes.

3.5 Backup

Módulo `Iter_Backup`, com restic. Cada cópia leva o banco em SQL puro (`pg_dump --no-owner --no-privileges`, em `var/backup/dump/banco.sql`, apagado depois), o `commit.txt` do código, `public/media` e `src/etc`. O restic compacta (zstd, nível máximo), cifra e só guarda o que mudou: com o banco de 17 MB e 4,5 MB de imagens, a primeira cópia ocupa 2,6 MB e cada nova cerca de 0,1 MB.

- **Configurações → Backup** (`system/backup/*`, permissão `Iter_Backup::backup`): ligado, intervalo (1 h a 1 semana, com horário e dia), retenção (`--keep-hourly/daily/weekly/monthly`, padrão 24/7/4/12; todos em 0 é recusado), e-mail de falha, avisos no Dashboard (`system/backup/dashboard_warnings`; desligado, o Dashboard não mostra nada do backup) e destino: pasta local (padrão `var/backup/repo`, nunca em `public/`), Backblaze B2 (`b2:<bucket>:iterstore`) ou S3 compatível (`s3:https://<endpoint>/<bucket>/iterstore`). A chave secreta é gravada cifrada.
- **Senha do repositório:** gerada no primeiro uso (64 hex), gravada cifrada em `system/backup/repo_password`. Credenciais vão ao restic pelo ambiente (`RESTIC_PASSWORD`, `B2_ACCOUNT_*`, `AWS_*`), a senha do banco por `PGPASSWORD`; nada vai na linha de comando.
- **Cron:** a cada minuto, com `flock`. Copia quando vence o intervalo ou existe `var/backup/run-now` (botão "Fazer backup agora"). Depois de uma falha, espera 15 minutos. Limpeza (`forget --prune`) uma vez por dia; teste de restauração uma vez a cada 30 dias.
- **Situação:** `var/backup/status.json` (última cópia, cópias, espaço, erro, teste), lido por Sistema → Backup e pelo Dashboard. O painel nunca chama o restic.

- **Teste de restauração:** o `banco.sql` da última cópia é restaurado num banco `<banco>_verify` (o usuário do banco precisa de `CREATEDB`), conferido (todas as tabelas, e pedidos, clientes e produtos ao menos os contados antes do dump) e apagado.
- **Kit de recuperação** (Sistema → Backup → Abrir o kit): só o perfil Administrador, pede a senha de novo, fica no log de operações (`backup_kit`). Mostra a senha do restic e a `crypt_key` do `env.ini`. Guardar no gerenciador de senhas da empresa e impresso, nunca por e-mail.

```
php bin/backup.php                    # o que o cron roda
php bin/backup.php now antes-migracao # uma cópia agora, com marcação
php bin/backup.php verify             # teste de restauração agora
php bin/backup.php zip [cópia]        # .zip de uma cópia (latest) em var/backup/,
permissão 600
```

RESTAURAR NUM SERVIDOR NOVO

```
export RESTIC_REPOSITORY='b2:bucket:iterstore'   # ou s3:https://..., ou a pasta
export RESTIC_PASSWORD='...'                     # do kit de recuperação
export B2_ACCOUNT_ID='...' B2_ACCOUNT_KEY='...'  # ou AWS_ACCESS_KEY_ID /
AWS_SECRET_ACCESS_KEY
restic snapshots                                 # escolha a cópia (latest = a última)
restic restore latest --target /tmp/r
createdb loja
grep -v '^SET transaction_timeout' /tmp/r/<raiz>/var/backup/dump/banco.sql | psql -v
ON_ERROR_STOP=1 -d loja
git checkout $(cat /tmp/r/<raiz>/var/backup/dump/commit.txt)
cp -a /tmp/r/<raiz>/public/media public/ && cp -a /tmp/r/<raiz>/src/etc/. src/etc/
composer install && php bin/migrate.php up
```

`<raiz>` é o caminho do sistema no servidor antigo (o restic guarda caminhos absolutos). Confira o `crypt_key` do `src/etc/env.ini` restaurado com o do kit e ajuste `database.ini` para o banco novo. A linha `SET transaction_timeout` só existe em dumps de pg_dump 17+ e é desconhecida antes do PostgreSQL 17.

4. A API GraphQL

4.1 Endpoint e formato

Toda chamada é um `POST` para `/core/graphql/index` com um JSON `{"query": "...", "variables": {...}}`. A resposta é sempre HTTP 200 com `data` e/ou `errors`, exceto token ou chave inválidos (401), excesso de requisições (429) e chamada sem HTTPS com "Usar só HTTPS" ligado (403).

```
curl -s https://sua-loja/core/graphql/index \
  -H 'Content-Type: application/json' \
  -H 'X-Api-Key: itk_…'   # quando a chave é exigida \
  -d '{"query":"{ websites { id name isDefault } }"}'
```

```
[{"id": "15", "name": "Loja RJ", "isDefault": true}, {"id": "216", "name": "Loja SP",
"isDefault": false}]
```

COM VARIÁVEIS (JAVASCRIPT)

```
const API = 'https://sua-loja/core/graphql/index';

async function gql(query, variables = {}, token = null) {
  const res = await fetch(API, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'X-Api-Key': process.env.API_KEY,   // Sistema → Chaves da API
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
    },
    body: JSON.stringify({ query, variables }),
  });
  const json = await res.json();
  if (json.errors?.length) {
    const err = new Error(json.errors[0].message);
    err.code = json.errors[0].extensions?.code;   // decida pelo código, não pela mensagem
    throw err;
  }
  return json.data;
}

const { product } = await gql(
  'query P($sku: String!) { product(sku: $sku) { name priceRange { min max } } }',
  { sku: 'MODA-CALCA-JEANS' },
);
```

EM PHP (GUZZLE, JÁ INSTALADO NO PROJETO)

```
$http = new GuzzleHttp\Client(['base_uri' => 'https://sua-loja/']);
$res = $http->post('core/graphql/index', [
    'headers' => ['Authorization' => 'Bearer ' . $token], // com o token de admin, a chave
não é pedida
    'json' => [
        'query' => 'query ($s: OrderStatus) { orders(status: $s, pageSize: 50) { total
items { number total } } }',
```

```
        'variables' => ['s' => 'processing'],
    ],
]);
$body = json_decode((string) $res->getBody(), true);
if (isset($body['errors'])) {
    throw new RuntimeException($body['errors'][0]['extensions']['code'] . ': ' .
$body['errors'][0]['message']);
}
```

Para explorar a API, use `Sistema → API (GraphiQL)` no painel: autocompletar, documentação de cada campo e execução com a sessão do admin logado.

4.2 Autenticação

Quem	Como entra	Validade
Visitante	Sem token. O carrinho é identificado pelo `cartId` devolvido por `createCart`.	O carrinho some depois dos dias de Configurações → Vendas.
Cliente da loja	`generateCustomerToken` (ou `registerCustomer`) devolve um token. Envie `Authorization: Bearer <token>`. `refreshCustomerToken` renova; `revokeCustomerToken` sai.	Configurações → Clientes (padrão 720 h).
Aplicativo ou site	Uma chave criada em `Sistema → Chaves da API`, enviada no cabeçalho `X-Api-Key` de toda chamada. Exigida quando "Exigir chave do aplicativo" está ligado em Configurações → API; aí, sem ela, a resposta é HTTP 401 `API_KEY_INVALID`.	Até ser revogada no painel.
Admin / integração	`generateAdminToken` com o usuário e a senha de um usuário do painel. O token tem as mesmas permissões do perfil dele. `revokeAdminToken` sai.	Configurações → Segurança do admin (padrão 4 h). Trocar a senha invalida os tokens anteriores.

> **Usuário próprio para cada integração**
> Crie um usuário do painel para cada sistema integrado (ex.: "integracao-erp"), com um perfil que só tenha as permissões necessárias. Tudo o que ele fizer fica no log de operações com o nome dele.

Com a chave exigida, o visitante e o cliente precisam dela (o login do admin também: `generateAdminToken` vai com a chave). Já com o token de admin em mãos, ou logado no painel (GraphiQL), a chave não é pedida. Uma chave por aplicativo: se uma vazar, revogue só ela e publique o app com a nova.

Um token de cliente nunca passa pelos campos do admin (a resposta é `UNAUTHENTICATED`). Um token inválido ou vencido é recusado com HTTP 401 e o código `TOKEN_INVALID`, mesmo em

campos públicos: descarte-o e faça login de novo.

4.3 Cabeçalhos e CORS

Cabeçalho	Uso
`Content-Type: application/json`	Obrigatório (exceto no envio de arquivos). Com a sessão do painel (GraphiQL), outro tipo dá 415 `UNSUPPORTED_MEDIA_TYPE`.
`Authorization: Bearer <token>`	Cliente ou admin.
`X-Api-Key`	A chave do aplicativo ou site (Sistema → Chaves da API).
`X-Sales-Channel`	No `placeOrder`: o canal de venda gravado no pedido ("Site", "App mobile", "Instagram"…). Sem ele, "App mobile". Aparece nos relatórios de canal.

Para chamar a API pelo navegador a partir de outro domínio, cadastre o site em Configurações → Geral → "Sites que podem usar a API pelo navegador (CORS)" (um por linha, ou `*`). Apps nativos e servidores não precisam de CORS. O CORS só controla navegadores: contra programas em servidores, valem a chave do aplicativo e os limites por IP. O login pela API nunca usa cookie: só o token.

4.4 Erros

Todo erro traz `extensions.code`. A mensagem é em português e pode ser mostrada ao usuário, mas a lógica do cliente deve decidir pelo código. Exemplo, um produto com grade colocado no carrinho:

```
{
  "errors": [
    {
      "message": "'Calça Jeans Skinny' tem grade: escolha a variação (cor, tamanho...).",
      "path": [
        "addToCart"
      ],
      "extensions": {
        "code": "PRODUCT_IS_CONFIGURABLE"
      }
    }
  ]
}
```

Erros de sintaxe e de campo inexistente (testados no servidor):

```
{"errors":[{"message":"Syntax Error: Expected Name, found <EOF>","extensions":
{"code":"GRAPHQL_PARSE_FAILED"}}]}
{"errors":[{"message":"Cannot query field \"slug\" on type \"Website\".","extensions":
{"code":"GRAPHQL_VALIDATION_FAILED"}}]}
```

Token inválido (HTTP 401):

```
{"errors":[{"message":"Token inválido ou expirado. Faça login de novo.","extensions":
{"code":"TOKEN_INVALID"}}]}
```

A lista dos códigos está no capítulo 9.

4.5 Limites

Limite	Padrão	Onde muda
Profundidade da consulta	15 níveis	Configurações → API
Complexidade da consulta	1.500 pontos sem login de admin; o admin, sem limite. Cada campo conta 1; uma lista paginada (`products`, `orders`, `myOrders`, `customers`…) conta os campos de dentro × o `pageSize` (até 100); o `report`, × `pageSize` ÷ 10. A consulta mais cara do app (`myOrders` com 20 por página) fica perto de 660	Configurações → API
Tamanho da chamada (corpo JSON)	64 KB sem login de admin, 8 MB com; acima, HTTP 413 `PAYLOAD_TOO_LARGE`. O envio de arquivos segue o limite do PHP	Configurações → API
Introspecção (`__schema`) sem login de admin	desligada (o admin logado sempre tem, para o GraphiQL)	Configurações → API
Requisições à API por IP (o admin não entra)	240 por minuto; acima, HTTP 429 `TOO_MANY_REQUESTS`	Configurações → API
Cotações de frete (`shippingQuotes`) por IP	20 por minuto	Configurações → API
Carrinhos criados / pedidos fechados / consultas de pedido de visitante, por IP	30 / 10 / 10 por minuto	Configurações → Vendas
Login do admin (API e painel) e "Esqueci a senha"	10 por minuto por IP	Configurações → Segurança do admin
Login e cadastro de cliente	10 por minuto por IP	Configurações → Clientes
Mensagens do cliente (`sendCustomerMessage`)	10 por minuto por cliente; as respostas da loja não têm limite	Configurações → Clientes
`pageSize` das listas	padrão 20; relatórios até 1.000	—

Os limites por IP contam um endereço IPv6 pelo bloco `/64` (quem tem um bloco inteiro não escapa trocando de endereço); o IPv4 conta pelo endereço.

Uma consulta acima dos limites é recusada antes de rodar:

```
{"errors":[{"message":"Max query depth should be 15 but got 20.","extensions":
{"code":"QUERY_LIMIT_EXCEEDED"}}]}
```

4.6 Envio de arquivos

`uploadProductImage` e `setCategoryBanners` recebem arquivos pelo *GraphQL multipart request spec* (`multipart/form-data` com `operations`, `map` e o arquivo). Exige o token do admin no cabeçalho: com só o cookie, o envio é recusado (proteção contra CSRF).

```
curl https://sua-loja/core/graphql/index \
  -H "Authorization: Bearer $TOKEN" \
  -F operations='{"query":"mutation ($f: Upload!) { uploadProductImage(productId:
\"1918\", code: \"imagem\", file: $f) { sku } }","variables":{"f":null}}' \
  -F map='{"0":["variables.f"]}' \
  -F 0=@foto.jpg
```

5. Exemplos: loja e catálogo

Campos públicos, sem token: é o que o app e o site usam para montar as telas. O produto inativo e os atributos ocultos não aparecem. `websiteId` omitido = o website padrão.

WEBSITES E DADOS DA LOJA

Os dados da loja (nome, telefone, endereço) vêm de Configurações → Loja; aqui eles ainda não foram preenchidos.

REQUISIÇÃO

```
{ websites { id name isDefault } storeInfo { name email phone address { city region } } }
```

RESPOSTA


```
{
  "data": {
    "websites": [
      {
        "id": "15",
        "name": "Loja RJ",
        "isDefault": true
      },
      {
        "id": "216",
        "name": "Loja SP",
        "isDefault": false
      }
    ],
    "storeInfo": {
      "name": null,
      "email": null,
      "phone": null,
      "address": {
        "city": null,
        "region": null
      }
    }
  }
}
```


ÁRVORE DE CATEGORIAS

Só as ativas do website, com as filhas em `children`.

REQUISIÇÃO

```
{ categories { id name url_key level children { id name url_key } } }
```

RESPOSTA


```
{
  "data": {
    "categories": [
      {
        "id": "13",
        "name": "Cosmético",
```

```
        "url_key": "cosmetico",
        "level": 0,
        "children": []
      },
      {
        "id": "224",
        "name": "Vestuário",
        "url_key": "vestuario",
        "level": 0,
        "children": [
          {
            "id": "538",
            "name": "Bermuda",
            "url_key": "bermuda"
          },
          {
            "id": "518",
            "name": "Calça",
            "url_key": "calca"
          },
          "… mais 7"
        ]
      },
      "… mais 6"
    ]
  }
}
```


BUSCA DE PRODUTOS COM PAGINAÇÃO

Busca por nome ou SKU. `aggregations` traz a faixa de preço e os valores dos atributos filtráveis do resultado, para montar os filtros da tela. Filtros por atributo: `filters: [{code: "color", values: ["azul"]}]`; por preço: `priceFrom` e `priceTo`.

REQUISIÇÃO

```
query Produtos($busca: String, $pagina: Int) {
  products(search: $busca, isActive: true, page: $pagina, pageSize: 3) {
    total
    items {
      id sku name type url_key
      priceRange { min max }
      stock { salable_quantity is_in_stock }
    }
    aggregations { price { min max } }
  }
}
```

VARIÁVEIS

```
{
  "busca": "blusa",
  "pagina": 1
}
```

RESPOSTA


```
{
  "data": {
    "products": {
```

```
      "total": 6,
      "items": [
        {
          "id": "1918",
          "sku": "MODA-BLUSA-CROPPED",
          "name": "Blusa Cropped Canelada",
          "type": "SIMPLE",
          "url_key": "blusa-cropped-canelada",
          "priceRange": {
            "min": 69.9,
            "max": 69.9
          },
          "stock": {
            "salable_quantity": 29,
            "is_in_stock": true
          }
        },
        {
          "id": "2029",
          "sku": "MODA-BLUSA-OMBRO",
          "name": "Blusa Ombro a Ombro",
          "type": "CONFIGURABLE",
          "url_key": "blusa-ombro-a-ombro",
          "priceRange": {
            "min": 89.9,
            "max": 89.9
          },
          "stock": {
            "salable_quantity": 48,
            "is_in_stock": true
          }
        },
        "… mais 1"
      ],
      "aggregations": {
        "price": {
          "min": 35.6,
          "max": 199.9
        }
      }
    }
  }
}
```


PRODUTO COM GRADE (VARIAÇÕES)

Um produto `CONFIGURABLE` traz os atributos da grade e cada variação com o seu SKU, estoque e opções. É o SKU da variação que vai para o carrinho.

REQUISIÇÃO

```
query Produto($sku: String!) {
  product(sku: $sku) {
    sku name type
    priceRange { min max }
    variantAttributes { code label }
    variants { sku name stock { salable_quantity } attributes { code option_label } }
  }
}
```

VARIÁVEIS

```
{
  "sku": "MODA-CALCA-JEANS"
}
```

RESPOSTA

```
{
  "data": {
    "product": {
      "sku": "MODA-CALCA-JEANS",
      "name": "Calça Jeans Skinny",
      "type": "CONFIGURABLE",
      "priceRange": {
        "min": 159.9,
        "max": 169.9
      },
      "variantAttributes": [
        {
          "code": "color",
          "label": "Cor"
        },
        {
          "code": "tamanho",
          "label": "Tamanho"
        }
      ],
      "variants": [
        {
          "sku": "MODA-CALCA-JEANS-AZUL-P",
          "name": "Calça Jeans Skinny - Azul / P",
          "stock": {
            "salable_quantity": 0
          },
          "attributes": [
            {
              "code": "color",
              "option_label": "Azul"
            },
            {
              "code": "tamanho",
              "option_label": "P"
            },
            "… mais 2"
          ]
        },
        {
          "sku": "MODA-CALCA-JEANS-AZUL-M",
          "name": "Calça Jeans Skinny - Azul / M",
          "stock": {
            "salable_quantity": 7
          },
          "attributes": [
            {
              "code": "color",
              "option_label": "Azul"
            },
            {
              "code": "tamanho",
              "option_label": "M"
```

```
            },
            "… mais 2"
          ]
        },
        "… mais 6"
      ]
    }
  }
}
```


RESOLVER UM ENDEREÇO DA LOJA

Para o site: o que abre em `/calca-jeans-skinny` (produto ou categoria).

REQUISIÇÃO


```
{ route(url: "/calca-jeans-skinny") { type product { sku name } category { name } } }
```


RESPOSTA


```
{
  "data": {
    "route": {
      "type": "PRODUCT",
      "product": {
        "sku": "MODA-CALCA-JEANS",
        "name": "Calça Jeans Skinny"
      },
      "category": null
    }
  }
}
```

6. Exemplos: compra do visitante

A sequência completa, do carrinho ao pedido. O cliente logado faz o mesmo sem `cartId` (usa o carrinho dele) e sem `setCartGuestInfo`.

1. `createCart` → guarde o `cartId` (no app, no armazenamento local).
2. `addToCart` / `updateCartItem` / `removeCartItem`.
3. `setCartGuestInfo` (e-mail, nome, CPF/CNPJ).
4. `setCartShippingAddress` → `availableShippingMethods` → `setCartShippingMethod`.
5. `availablePaymentMethods` → `setCartPaymentMethod`. Cupom: `applyCartCoupon`.
6. `placeOrder` com o `expectedTotal` que o cliente viu.
7. Mercado Pago: `payOrder` (Pix: QR Code; cartão: token do SDK).

1. CRIAR O CARRINHO

REQUISIÇÃO

```
mutation { createCart }
```

RESPOSTA

```
{
  "data": {
    "createCart": "31312e08…1183"
  }
}
```

2. ADICIONAR UM PRODUTO

`quantity` soma ao que já estiver no carrinho. Cada item traz `error` quando algo mudou (sem estoque, sem preço): `hasErrors` impede o pedido.

REQUISIÇÃO

```
mutation Adicionar($cartId: String!, $sku: String!) {
  addToCart(cartId: $cartId, sku: $sku, quantity: 2) { items { id sku name quantity price
rowTotal error } subtotal discountAmount shippingAmount total hasErrors }
}
```

VARIÁVEIS

```
{
  "cartId": "31312e08…1183",
  "sku": "001"
}
```

RESPOSTA

```
{
  "data": {
    "addToCart": {
      "items": [
        {
```

```
          "id": "4991",
          "sku": "001",
          "name": "Produto teste",
          "quantity": 2,
          "price": 8.5,
          "rowTotal": 17,
          "error": null
        }
      ],
      "subtotal": 17,
      "discountAmount": 0,
      "shippingAmount": 0,
      "total": 17,
      "hasErrors": false
    }
  }
}
```


ERRO: PRODUTO COM GRADE

Enviar o SKU do produto pai é recusado; envie o da variação.

REQUISIÇÃO

```
mutation { addToCart(cartId:
"31312e081da812baa6fc25c5b2ea38858978bf706a0c229f8ffe5ff05e291183", sku: "MODA-CALCA-
JEANS", quantity: 1) { total } }
```

RESPOSTA


```
{
  "errors": [
    {
      "message": "'Calça Jeans Skinny' tem grade: escolha a variação (cor, tamanho...).",
      "path": [
        "addToCart"
      ],
      "extensions": {
        "code": "PRODUCT_IS_CONFIGURABLE"
      }
    }
  ]
}
```


3. DADOS DO VISITANTE

O CPF/CNPJ é validado e guardado só com os dígitos.

REQUISIÇÃO

```
mutation Visitante($cartId: String!) {
  setCartGuestInfo(cartId: $cartId, email: "maria@exemplo.com.br", firstname: "Maria",
lastname: "Souza", taxvat: "529.982.247-25") { email firstname taxvat }
}
```

VARIÁVEIS

```
{
  "cartId": "31312e08…1183"
}
```

RESPOSTA

```
{
  "data": {
    "setCartGuestInfo": {
      "email": "maria@exemplo.com.br",
      "firstname": "Maria",
      "taxvat": "52998224725"
    }
  }
}
```

COTAÇÃO DE FRETE ANTES DO ENDEREÇO

Para o "calcule o frete" da tela do produto ou do carrinho, só com o CEP.

REQUISIÇÃO

```
{ shippingQuotes(cartId:
"31312e081da812baa6fc25c5b2ea38858978bf706a0c229f8ffe5ff05e291183", destinationCep:
"20040-020") { code label amount days } }
```

RESPOSTA

```
{
  "data": {
    "shippingQuotes": [
      {
        "code": "pickup",
        "label": "Retirar na loja (Avenida Ayrton Senna, 2600, Rio de Janeiro)",
        "amount": 0,
        "days": 1
      }
    ]
  }
}
```

4. ENDEREÇO DE ENTREGA E FORMAS DE ENTREGA

O CEP pode ir com ou sem máscara. As formas de entrega dependem do endereço e do carrinho (faixas de frete, Melhor Envio, retirada na loja).

REQUISIÇÃO

```
mutation Endereco($cartId: String!, $endereco: CartAddressInput!) {
  setCartShippingAddress(cartId: $cartId, address: $endereco) {
    shippingAddress { postcode city region }
    availableShippingMethods { code label amount days }
  }
}
```

VARIÁVEIS

```
{
  "cartId": "31312e08…1183",
  "endereco": {
    "recipient": "Maria Souza",
    "postcode": "20040-020",
    "street": "Avenida Rio Branco",
    "number": "1",
```

```
    "district": "Centro",
    "city": "Rio de Janeiro",
    "region": "RJ",
    "telephone": "(21) 99999-0000"
  }
}
```

RESPOSTA

```
{
  "data": {
    "setCartShippingAddress": {
      "shippingAddress": {
        "postcode": "20040020",
        "city": "Rio de Janeiro",
        "region": "RJ"
      },
      "availableShippingMethods": [
        {
          "code": "pickup",
          "label": "Retirar na loja (Avenida Ayrton Senna, 2600, Rio de Janeiro)",
          "amount": 0,
          "days": 1
        }
      ]
    }
  }
}
```

ESCOLHER A ENTREGA

Depois da entrega, `availablePaymentMethods` lista as formas de pagamento válidas para este carrinho (nesta loja, as do Mercado Pago).

REQUISIÇÃO

```
mutation { setCartShippingMethod(cartId:
"31312e081da812baa6fc25c5b2ea38858978bf706a0c229f8ffe5ff05e291183", code: "pickup") {
shippingMethod { code label amount } total availablePaymentMethods { code label } } }
```

RESPOSTA

```
{
  "data": {
    "setCartShippingMethod": {
      "shippingMethod": {
        "code": "pickup",
        "label": "Retirar na loja (Avenida Ayrton Senna, 2600, Rio de Janeiro)",
        "amount": 0
      },
      "total": 17,
      "availablePaymentMethods": [
        {
          "code": "mercadopago_pix",
          "label": "Pix"
        },
        {
          "code": "mercadopago_credit",
          "label": "Cartão de crédito"
        },
```

```
        {
          "code": "mercadopago_debit",
          "label": "Cartão de débito"
        }
      ]
    }
  }
}
```

5. ESCOLHER O PAGAMENTO

REQUISIÇÃO

```
mutation { setCartPaymentMethod(cartId:
"31312e081da812baa6fc25c5b2ea38858978bf706a0c229f8ffe5ff05e291183", code:
"mercadopago_pix") { paymentMethod { code label } total } }
```

RESPOSTA

```
{
  "data": {
    "setCartPaymentMethod": {
      "paymentMethod": {
        "code": "mercadopago_pix",
        "label": "Pix"
      },
      "total": 17
    }
  }
}
```

ERRO: O TOTAL MUDOU

`expectedTotal` protege o cliente de pagar um valor diferente do que viu (preço, promoção ou frete mudaram no meio). Mostre o carrinho de novo e peça a confirmação.

REQUISIÇÃO

```
mutation { placeOrder(cartId:
"31312e081da812baa6fc25c5b2ea38858978bf706a0c229f8ffe5ff05e291183", expectedTotal: 1.00) {
number } }
```

RESPOSTA

```
{
  "errors": [
    {
      "message": "O total do carrinho mudou. Revise os valores atualizados e confirme
novamente.",
      "path": [
        "placeOrder"
      ],
      "extensions": {
        "code": "TOTAL_CHANGED"
      }
    }
  ]
}
```

6. FECHAR O PEDIDO

Reserva o estoque, envia o e-mail "pedido recebido" e devolve o pedido. Chamar de novo com o mesmo carrinho devolve o mesmo pedido (seguro contra clique duplo ou falha de rede). O canal veio do cabeçalho `X-Sales-Channel`.

REQUISIÇÃO

`X-Sales-Channel: Site`

```
mutation Fechar($cartId: String!, $total: Float!) {
  placeOrder(cartId: $cartId, expectedTotal: $total) {
    id number status statusLabel total salesChannel
    paymentMethod paymentInstructions
    items { sku name quantity price rowTotal }
  }
}
```

VARIÁVEIS

```
{
  "cartId": "31312e08…1183",
  "total": 17
}
```

RESPOSTA

```
{
  "data": {
    "placeOrder": {
      "id": "4838",
      "number": "RJ000060",
      "status": "pending_payment",
      "statusLabel": "Aguardando pagamento",
      "total": 17,
      "salesChannel": "Site",
      "paymentMethod": "mercadopago_pix",
      "paymentInstructions": null,
      "items": [
        {
          "sku": "001",
          "name": "Produto teste",
          "quantity": 2,
          "price": 8.5,
          "rowTotal": 17
        }
      ]
    }
  }
}
```

ACOMPANHAR O PEDIDO DO VISITANTE

Com o número e o e-mail do pedido (a tela "acompanhe seu pedido").

REQUISIÇÃO

```
{ guestOrder(number: "RJ000060", email: "maria@exemplo.com.br") { number statusLabel total
} }
```

RESPOSTA

```
{
  "data": {
    "guestOrder": {
      "number": "RJ000060",
      "statusLabel": "Aguardando pagamento",
      "total": 17
    }
  }
}
```

7. PAGAR PELO MERCADO PAGO

`payOrder` cobra um pedido "Aguardando pagamento". No Pix, devolve o copia e cola (`pixQrCode`), a imagem (`pixQrCodeBase64`) e a validade; chamar de novo devolve o mesmo QR enquanto ele valer. No cartão, o app gera o token com o SDK MercadoPago.js V2 (a `publicKey` vem de `mercadopagoSettings`) e envia em `card`; recusado, pode tentar de novo com outro cartão. Este exemplo não foi executado aqui porque cria uma cobrança real no Mercado Pago.

- `card.paymentMethodId` é o que o SDK devolveu em `getInstallments`. No débito, um cartão de débito vem como `debvisa`, `debmaster`, `debelo`… e é cobrado como `debit_card` em 1x; um pré-pago vem com a bandeira simples (`elo`, `visa`…) e é cobrado como `prepaid_card`, sem parcelas.
- `deviceId`: o identificador do aparelho que o `security.js` do Mercado Pago gera (`MP_DEVICE_SESSION_ID`). Vai no cabeçalho `X-meli-session-id` e melhora a aprovação. Opcional.
- Cada cobrança leva, além do valor: os itens (SKU, nome, opções, quantidade, preço e a "Categoria dos produtos" das Configurações), o frete como um item "Frete" (a soma dos itens precisa bater com o total; com desconto, o pedido vai como um item só), nome, CPF/CNPJ, e-mail, telefone e endereço do pagador, o "Nome na fatura do cartão" e, para cliente logado, a data do cadastro, se é a primeira compra e a data da última.
- No app Jardim Vera, o SDK roda numa WebView invisível (`react-native-webview`): bandeira, parcelas, token e `deviceId` vêm do próprio MercadoPago.js. O número do cartão nunca passa pelo servidor da loja.

```
mutation {
  payOrder(number: "RJ000056", email: "maria@exemplo.com.br") {
    status statusLabel amount
    pixQrCode pixQrCodeBase64 expiresAt
    order { status statusLabel }
  }
}

# cartão
mutation ($card: MercadopagoCardInput!, $deviceId: String) {
  payOrder(number: "RJ000056", email: "maria@exemplo.com.br", card: $card, deviceId:
$deviceId) {
    status statusDetail cardBrand cardNumber installments
    challengeUrl challengeCreq
    order { status }
  }
}
```

```
# variables: { "card": { "token": "<token do SDK>", "paymentMethodId": "visa",
"installments": 3 }, "deviceId": "<MP_DEVICE_SESSION_ID>" }
```

A confirmação chega pelo webhook (e pelo cron de sincronização): o pedido passa para "Pagamento confirmado" (`processing`). O app consulta o pedido (`guestOrder` ou `myOrder`) para mostrar a situação.

7. Exemplos: conta do cliente

CADASTRO

Devolve o token já logado. Com "Exigir confirmação do e-mail" ligado, `token` vem nulo e `confirmationRequired: true`: o cliente confirma pelo link do e-mail (`confirmCustomerEmail`). Para pessoa jurídica: `type: PJ`, CNPJ, `companyName` e inscrição estadual.

REQUISIÇÃO

```
mutation Cadastro($senha: String!) {
  registerCustomer(email: "joao@exemplo.com.br", firstname: "João", lastname: "Lima",
type: PF, taxvat: "111.444.777-35", password: $senha) {
    token expires_at confirmationRequired
    customer { id email firstname }
  }
}
```

VARIÁVEIS

```
{
  "senha": "SenhaDoJoao123"
}
```

RESPOSTA

```
{
  "data": {
    "registerCustomer": {
      "token": "51e3fbb2…e66f",
      "expires_at": "2026-11-19 18:08:28",
      "confirmationRequired": false,
      "customer": {
        "id": "2668",
        "email": "joao@exemplo.com.br",
        "firstname": "João"
      }
    }
  }
}
```

LOGIN

REQUISIÇÃO

```
mutation {
  generateCustomerToken(email: "joao@exemplo.com.br", password: "SenhaDoJoao123") { token
expires_at customer { id firstname } }
}
```

RESPOSTA

```
{
  "data": {
    "generateCustomerToken": {
      "token": "9bc77bc4…83f9",
      "expires_at": "2026-11-19 18:08:28",
      "customer": {
```

```
        "id": "2668",
        "firstname": "João"
      }
    }
  }
}
```

DADOS DO CLIENTE LOGADO

REQUISIÇÃO

`Authorization: Bearer <token>`

```
{ me { id email firstname lastname type taxvat addresses { id city is_default_shipping } }
}
```

RESPOSTA

```
{
  "data": {
    "me": {
      "id": "2668",
      "email": "joao@exemplo.com.br",
      "firstname": "João",
      "lastname": "Lima",
      "type": "PF",
      "taxvat": "11144477735",
      "addresses": []
    }
  }
}
```

SEM TOKEN

`me` sem login devolve `null` (não é erro): o app sabe que precisa pedir o login.

REQUISIÇÃO

```
{ me { id } }
```

RESPOSTA

```
{
  "data": {
    "me": null
  }
}
```

CADASTRAR ENDEREÇO

Devolve a lista de endereços. O endereço padrão de entrega pode ser usado no carrinho com `setCartShippingAddress(customerAddressId: ...)`.

REQUISIÇÃO

`Authorization: Bearer <token>`

```
mutation {
  createMyAddress(address: { recipient: "João Lima", postcode: "01310-100", street:
"Avenida Paulista", number: "1000", district: "Bela Vista", city: "São Paulo", region:
"SP", isDefaultShipping: true, isDefaultBilling: true }) {
```

```
    id city region is_default_shipping
  }
}
```

RESPOSTA

```
{
  "data": {
    "createMyAddress": [
      {
        "id": "2420",
        "city": "São Paulo",
        "region": "SP",
        "is_default_shipping": true
      }
    ]
  }
}
```

CARRINHO DO CLIENTE

Sem `cartId`: o cliente tem um carrinho por website, que acompanha o login em qualquer aparelho.

REQUISIÇÃO

```
Authorization: Bearer <token>
```

```
{ cart { isGuest email items { sku quantity } total } }
```

RESPOSTA

```
{
  "data": {
    "cart": {
      "isGuest": false,
      "email": "joao@exemplo.com.br",
      "items": [],
      "total": 0
    }
  }
}
```

JUNTAR O CARRINHO DE VISITANTE AO FAZER LOGIN

O visitante montou um carrinho e depois entrou: `mergeCarts` com o `cartId` antigo leva os itens para o carrinho do cliente.

REQUISIÇÃO

```
Authorization: Bearer <token>
```

```
mutation { mergeCarts(cartId:
"f2cb98dced50ce2741a960854799bba2ba2a0e14e842d2839dca79a22cbe6eeb") { isGuest items { sku
quantity } total } }
```

RESPOSTA

```
{
  "data": {
    "mergeCarts": {
```

```
      "isGuest": false,
      "items": [
        {
          "sku": "001",
          "quantity": 1
        }
      ],
      "total": 8.5
    }
  }
}
```

MEUS PEDIDOS

Paginado. `myOrder(number:)` traz um pedido com itens, endereço, rastreio e o histórico visível ao cliente.

REQUISIÇÃO

`Authorization: Bearer <token>`

```
{ myOrders { total items { number statusLabel total } } }
```

RESPOSTA

```
{
  "data": {
    "myOrders": {
      "total": 0,
      "items": []
    }
  }
}
```

8. Exemplos: integração do admin

Para ERP, aplicativo de atendimento, contabilidade e automações. Tudo com o token do admin; cada ação grava no histórico do pedido e no log de operações o usuário do token.

8.1 Login

GERAR O TOKEN

REQUISIÇÃO

```
mutation {
  generateAdminToken(username: "integracao", password: "SenhaForte123") {
    token
    expires_at
    user { username role }
  }
}
```

RESPOSTA

```
{
  "data": {
    "generateAdminToken": {
      "token": "0532c3e0…9773",
      "expires_at": "2026-10-20 22:08:27",
      "user": {
        "username": "integracao",
        "role": "Administradores"
      }
    }
  }
}
```

SENHA ERRADA

A mesma mensagem para usuário inexistente, senha errada ou conta bloqueada (não revela quais usuários existem). Depois do limite por IP, o código vira `TOO_MANY_REQUESTS` (HTTP 429).

REQUISIÇÃO

```
mutation { generateAdminToken(username: "integracao", password: "errada") { token } }
```

RESPOSTA

```
{
  "errors": [
    {
      "message": "Usuário ou senha inválidos, ou conta bloqueada temporariamente.",
      "path": [
        "generateAdminToken"
      ],
      "extensions": {
        "code": "INVALID_CREDENTIALS"
      }
    }
```

```
  ]
}
```

QUEM É O TOKEN

REQUISIÇÃO

`Authorization: Bearer <token>`

```
{ currentAdmin { id username firstname role } }
```

RESPOSTA

```
{
  "data": {
    "currentAdmin": {
      "id": "4503",
      "username": "integracao",
      "firstname": "Integração",
      "role": "Administradores"
    }
  }
}
```

TOKEN DE CLIENTE NUM CAMPO DO ADMIN

O token de cliente nunca vale como admin.

REQUISIÇÃO

`Authorization: Bearer <token>`

```
{ orders { total } }
```

RESPOSTA

```
{
  "errors": [
    {
      "message": "Acesso negado.",
      "path": [
        "orders"
      ],
      "extensions": {
        "code": "UNAUTHENTICATED"
      }
    }
  ]
}
```

8.2 Pedidos

LISTAR PEDIDOS

Filtros: `status`, `websiteId`, `customerId`, `salesChannel`, `paymentMethod`, `cancelReason`, `search` (número, nome, e-mail, CPF), `from`/`to` (AAAA-MM-DD), ordenação e página. `amount` é a soma do resultado.

REQUISIÇÃO

```
Authorization: Bearer <token>
```

```
{
  orders(status: pending_payment, page: 1, pageSize: 2) {
    total amount
    items { number createdAt statusLabel total salesChannel paymentLabel }
  }
}
```

RESPOSTA


```
{
  "data": {
    "orders": {
      "total": 2,
      "amount": 30.5,
      "items": [
        {
          "number": "RJ000060",
          "createdAt": "2026-10-20 18:08:27",
          "statusLabel": "Aguardando pagamento",
          "total": 17,
          "salesChannel": "Site",
          "paymentLabel": "Pix"
        },
        {
          "number": "RJ000057",
          "createdAt": "2026-10-20 12:12:44",
          "statusLabel": "Aguardando pagamento",
          "total": 13.5,
          "salesChannel": "App mobile",
          "paymentLabel": "Pix"
        }
      ]
    }
  }
}
```


CONFIRMAR O PAGAMENTO

Aguardando pagamento → Pagamento confirmado. Avisa o cliente por e-mail. Exige "Pedidos: confirmar pagamento".

REQUISIÇÃO

```
Authorization: Bearer <token>
```

```
mutation { confirmOrderPayment(id: "4838", comment: "Pix conferido no extrato") { number
status statusLabel history { status comment createdAt } } }
```

RESPOSTA


```
{
  "data": {
    "confirmOrderPayment": {
      "number": "RJ000060",
      "status": "processing",
      "statusLabel": "Pagamento confirmado",
      "history": [
        {
          "status": "processing",
```

```
          "comment": "Pix conferido no extrato",
          "createdAt": "2026-10-20 18:08:27"
        },
        {
          "status": "pending_payment",
          "comment": null,
          "createdAt": "2026-10-20 18:08:27"
        }
      ]
    }
  }
}
```

REGISTRAR O ENVIO

Pagamento confirmado → Enviado: baixa o estoque da origem (a sugerida, se `sourceCode` for omitido) e manda o rastreio ao cliente. Depois, `completeOrder` marca como Concluído.

REQUISIÇÃO

```
Authorization: Bearer <token>
```

```
mutation { shipOrder(id: "4838", carrier: "Correios", trackingCode: "AA123456789BR") {
number statusLabel shipment { carrier trackingCode } } }
```

RESPOSTA

```
{
  "data": {
    "shipOrder": {
      "number": "RJ000060",
      "statusLabel": "Enviado",
      "shipment": {
        "carrier": "Correios",
        "trackingCode": "AA123456789BR"
      }
    }
  }
}
```

CANCELAR

```
mutation {
  cancelOrder(id: "3947", reason: OUT_OF_STOCK, comment: "Sem a cor azul", notify: true,
refund: true) {
    number status cancelReason history { status comment }
  }
}
```

Motivos: `CUSTOMER`, `UNPAID`, `OUT_OF_STOCK`, `FRAUD`, `ORDER_ERROR`, `OTHER` (este exige `comment`). Pedido pago pelo Mercado Pago é estornado lá com `refund: true`; pago de outra forma exige `refundNote` (como o dinheiro foi devolvido). A reserva de estoque volta.

RESUMO DE VENDAS POR SITUAÇÃO

Quantos pedidos e quanto em cada situação, no período.

REQUISIÇÃO

```
Authorization: Bearer <token>
```

```
{ salesSummary(from: "2026-10-01", to: "2026-10-20") { status statusLabel count amount } }
```

RESPOSTA


```
{
  "data": {
    "salesSummary": [
      {
        "status": "pending_payment",
        "statusLabel": "Aguardando pagamento",
        "count": 1,
        "amount": 13.5
      },
      {
        "status": "processing",
        "statusLabel": "Pagamento confirmado",
        "count": 30,
        "amount": 879.55
      },
      {
        "status": "shipped",
        "statusLabel": "Enviado",
        "count": 1,
        "amount": 17
      },
      {
        "status": "complete",
        "statusLabel": "Concluído",
        "count": 3,
        "amount": 193.95
      },
      {
        "status": "canceled",
        "statusLabel": "Cancelado",
        "count": 25,
        "amount": 943.16
      }
    ]
  }
}
```


8.3 Produtos e estoque

OS ATRIBUTOS DO PRODUTO

Os códigos que podem ir em `attributes` ao criar ou alterar um produto. O admin vê também os ocultos.

REQUISIÇÃO

`Authorization: Bearer <token>`

```
{ attributes(entityType: "product") { code label inputType isRequired isFilterable } }
```

RESPOSTA


```
{
  "data": {
    "attributes": [
```

```
      {
        "code": "color",
        "label": "Cor",
        "inputType": "select",
        "isRequired": false,
        "isFilterable": false
      },
      {
        "code": "imagem",
        "label": "Imagem",
        "inputType": "image",
        "isRequired": false,
        "isFilterable": false
      },
      {
        "code": "imagem3",
        "label": "imagem3",
        "inputType": "image",
        "isRequired": false,
        "isFilterable": false
      },
      {
        "code": "tamanho",
        "label": "Tamanho",
        "inputType": "select",
        "isRequired": true,
        "isFilterable": true
      },
      "… mais 8"
    ]
  }
}
```

CRIAR UM PRODUTO

Preço, custo, peso e os outros campos vão em `attributes`, como texto (decimal com ponto). Precisa de ao menos uma categoria de cada website. O custo exige a permissão "Ver e editar o custo" e nunca aparece na API pública. Produto com grade: `type: CONFIGURABLE` com `variantAttributes`, e depois `saveVariants`.

REQUISIÇÃO

`Authorization: Bearer <token>`

```
mutation Novo($attrs: [ProductAttributeInput!]) {
  createProduct(sku: "CANECA-001", name: "Caneca de cerâmica", attributeSetId: "1",
websiteIds: ["15"], categoryIds: ["22"], attributes: $attrs) {
    id sku name url_key is_active
    attributes { code value }
  }
}
```

VARIÁVEIS

```
{
  "attrs": [
    {
      "code": "price",
      "value": "49.90"
    },
```

```
    {
      "code": "cost",
      "value": "18.50"
    },
    {
      "code": "weight",
      "value": "0.4"
    }
  ]
}
```

RESPOSTA

```
{
  "data": {
    "createProduct": {
      "id": "14668",
      "sku": "CANECA-001",
      "name": "Caneca de cerâmica",
      "url_key": "caneca-de-ceramica",
      "is_active": true,
      "attributes": [
        {
          "code": "price",
          "value": "49.9"
        },
        {
          "code": "cost",
          "value": "18.5"
        },
        {
          "code": "weight",
          "value": "0.4"
        }
      ]
    }
  }
}
```

DEFINIR O ESTOQUE

A quantidade física por origem (código da origem em Catálogo → Estoque). O vendável desconta as reservas dos pedidos ainda não enviados.

REQUISIÇÃO

`Authorization: Bearer <token>`

```
mutation { setProductStock(sku: "CANECA-001", sources: [{ code: "default", quantity: 25
}]) { salable_quantity is_in_stock sources { code quantity } } }
```

RESPOSTA

```
{
  "data": {
    "setProductStock": {
      "salable_quantity": 25,
      "is_in_stock": true,
      "sources": [
        {
          "code": "default",
```

```
          "quantity": 25
        }
      ]
    }
  }
}
```

ALTERAR O PREÇO

Só os campos enviados mudam. A alteração entra no log de operações com antes e depois. Para muitos custos de uma vez, há também a importação por planilha no painel.

REQUISIÇÃO

```
Authorization: Bearer <token>
```

```
mutation { updateProduct(id: "14668", attributes: [{ code: "price", value: "44.90" }]) {
sku attributes { code value } } }
```

RESPOSTA

```
{
  "data": {
    "updateProduct": {
      "sku": "CANECA-001",
      "attributes": [
        {
          "code": "price",
          "value": "44.9"
        },
        {
          "code": "cost",
          "value": "18.5"
        },
        {
          "code": "weight",
          "value": "0.4"
        }
      ]
    }
  }
}
```

8.4 Relatórios

OS RELATÓRIOS DISPONÍVEIS

Só os que o perfil do token pode abrir, com os filtros de cada um.

REQUISIÇÃO

```
Authorization: Bearer <token>
```

```
{ reports { code groupLabel title filters { key type default options { value label } } } }
```

RESPOSTA

```
{
  "data": {
    "reports": [
```

```
    {
      "code": "v8",
      "groupLabel": "Vendas",
      "title": "Promoções e cupons",
      "filters": [
        {
          "key": "promotion_id",
          "type": "select",
          "default": null,
          "options": [
            {
              "value": "",
              "label": "Todas as promoções"
            },
            {
              "value": "31",
              "label": "JARDIM10"
            }
          ]
        },
        {
          "key": "kind",
          "type": "select",
          "default": null,
          "options": [
            {
              "value": "",
              "label": "Automáticas e cupons"
            },
            {
              "value": "auto",
              "label": "Só automáticas"
            },
            {
              "value": "coupon",
              "label": "Só cupons"
            }
          ]
        },
        {
          "key": "q",
          "type": "search",
          "default": null,
          "options": null
        }
      ]
    },
    {
      "code": "v1",
      "groupLabel": "Vendas",
      "title": "Vendas por período",
      "filters": [
        {
          "key": "group_by",
          "type": "select",
          "default": null,
          "options": [
            {
              "value": "day",
              "label": "Dia"
            },
```

```
            {
              "value": "week",
              "label": "Semana"
            },
            {
              "value": "month",
              "label": "Mês"
            }
          ]
        },
        {
          "key": "basis",
          "type": "select",
          "default": null,
          "options": [
            {
              "value": "created",
              "label": "Data do pedido"
            },
            {
              "value": "paid",
              "label": "Data do pagamento"
            }
          ]
        }
      ]
    },
    {
      "code": "v2",
      "groupLabel": "Vendas",
      "title": "Vendas por produto",
      "filters": [
        {
          "key": "category_id",
          "type": "select",
          "default": null,
          "options": [
            {
              "value": "",
              "label": "Todas as categorias"
            },
            {
              "value": "13",
              "label": "Cosmético"
            },
            {
              "value": "224",
              "label": "Vestuário"
            },
            "… mais 16"
          ]
        },
        {
          "key": "q",
          "type": "search",
          "default": null,
          "options": null
        },
        {
          "key": "parent",
          "type": "checkbox",
```

```
            "default": "1",
            "options": null
          }
        ]
      },
      "… mais 23"
    ]
  }
}
```

UM RELATÓRIO

Os mesmos números da tela. `rows` traz uma lista por linha, na ordem de `columns`, com os valores sem formatação (o `type` diz como mostrar: `money`, `pct` como fração, `date`…). `tiles` são os indicadores já formatados. Consultar relatórios de clientes ou financeiros grava no log de operações.

REQUISIÇÃO

`Authorization: Bearer <token>`

```
query {
  report(code: "v1", filters: [{ key: "preset", value: "month" }, { key: "group_by",
value: "day" }], pageSize: 5) {
    columns { key label type }
    rows
    total
    totals
    tiles { label value note }
  }
}
```

RESPOSTA

```
{
  "data": {
    "report": {
      "columns": [
        {
          "key": "period",
          "label": "Dia",
          "type": "text"
        },
        {
          "key": "orders",
          "label": "Pedidos",
          "type": "int"
        },
        {
          "key": "items",
          "label": "Itens",
          "type": "qty"
        },
        "… mais 9"
      ],
      "rows": [
        [
          "01/10/2026",
          "0",
          "0",
```

```
          "… mais 9"
        ],
        [
          "02/10/2026",
          "0",
          "0",
          "… mais 9"
        ],
        [
          "03/10/2026",
          "0",
          "0",
          "… mais 9"
        ],
        "… mais 2"
      ],
      "total": 20,
      "totals": [
        null,
        "34",
        "73.0000",
        "… mais 9"
      ],
      "tiles": [
        {
          "label": "Faturamento",
          "value": "R$ 1.090,50",
          "note": "sem vendas nos 20 dias antes"
        },
        {
          "label": "Pedidos pagos",
          "value": "34",
          "note": "73 itens"
        },
        {
          "label": "Ticket médio",
          "value": "R$ 32,07",
          "note": "com frete"
        },
        "… mais 3"
      ]
    }
  }
}
```


SAIR

O token para de valer na hora.

REQUISIÇÃO

`Authorization: Bearer <token>`

```
mutation { revokeAdminToken }
```

RESPOSTA


```
{
  "data": {
    "revokeAdminToken": true
```

```
    }
}
```

9. Códigos de erro

Os mais comuns, por assunto. A descrição de cada campo no `schema.graphql` lista os códigos específicos dele.

COMUNS

Código	Quando
`UNAUTHENTICATED`	Sem login (ou com o token de outro tipo: um token de cliente num campo do admin).
`FORBIDDEN`	Logado, mas o perfil não tem a permissão.
`TOKEN_INVALID`	HTTP 401: o token enviado não existe, expirou ou foi revogado. Faça login de novo.
`API_KEY_INVALID`	HTTP 401: a chave do aplicativo é exigida e não veio, não existe ou foi revogada (cabeçalho X-Api-Key).
`TOO_MANY_REQUESTS`	Requisições, tentativas de login ou cotações de frete demais do mesmo IP no minuto (HTTP 429 no limite geral), ou mensagens demais do mesmo cliente.
`PAYLOAD_TOO_LARGE`	HTTP 413: o corpo da chamada passou do tamanho máximo (64 KB sem login de admin).
`UNSUPPORTED_MEDIA_TYPE`	HTTP 415: chamada feita com a sessão do painel (cookie) sem Content-Type: application/json. Protege o admin de um formulário de outro site.
`GRAPHQL_PARSE_FAILED`	A consulta tem erro de sintaxe.
`GRAPHQL_VALIDATION_FAILED`	Campo ou argumento que não existe, variável de tipo errado, ou introspecção desligada. Sem login de admin, a mensagem não traz as sugestões de nomes ("Did you mean...").
`QUERY_LIMIT_EXCEEDED`	A consulta passou da profundidade ou da complexidade máxima.
`NOT_FOUND`	O registro não existe ou não é visível para quem pediu.
`INTERNAL_ERROR`	Falha inesperada no servidor; a causa vai para o log do sistema.
`BAD_REQUEST`	Pedido inválido, sem código mais específico.

LOGIN E CADASTRO

Código	Quando
`INVALID_CREDENTIALS`	Usuário ou senha errados, ou conta bloqueada temporariamente.
`PASSWORD_EXPIRED`	A senha do admin venceu: troque pelo painel.

`EMAIL_NOT_CONFIRMED`	O cliente ainda não confirmou o e-mail.
`EMAIL_DUPLICATE`	E-mail já cadastrado no website.
`TAXVAT_INVALID`	CPF/CNPJ inválido.
`TAXVAT_DUPLICATE`	CPF/CNPJ já cadastrado no website.
`PASSWORD_INVALID`	Senha curta demais ou senha atual errada.

CATÁLOGO E ESTOQUE

Código	Quando
`PRODUCT_NOT_FOUND`	SKU ou id que não existe.
`SKU_DUPLICATE`	SKU já usado.
`CATEGORY_REQUIRED`	O produto precisa de uma categoria do website.
`URL_KEY_DUPLICATE`	Chave de URL já usada.
`ATTRIBUTE_NOT_FOUND`	Atributo fora do conjunto do produto.
`VALUE_INVALID`	Valor inválido para o atributo.
`VALUE_REQUIRED`	Atributo obrigatório vazio.
`SOURCE_NOT_FOUND`	Origem de estoque que não existe.

CARRINHO E PEDIDO

Código	Quando
`CART_NOT_FOUND`	cartId que não existe (ou de outro cliente).
`PRODUCT_IS_CONFIGURABLE`	Produto com grade: envie o SKU da variação.
`INSUFFICIENT_STOCK`	Quantidade acima do vendável.
`QUANTITY_INVALID`	Quantidade zero, negativa ou fracionada onde não pode.
`SHIPPING_CEP_INVALID`	CEP inválido.
`SHIPPING_METHOD_UNAVAILABLE`	Forma de entrega que não vale para o endereço/carrinho.
`PAYMENT_METHOD_UNAVAILABLE`	Forma de pagamento desligada ou fora do limite de valor.
`COUPON_NOT_FOUND`	Cupom que não existe.
`COUPON_EXPIRED`	Cupom fora da validade.
`COUPON_USED_UP`	Cupom sem usos restantes.
`COUPON_LOGIN_REQUIRED`	Cupom só para cliente logado.

`TOTAL_CHANGED`	O total mudou entre a revisão e o placeOrder (expectedTotal): mostre o novo total e peça a confirmação.
`ORDER_NOT_FOUND`	Pedido que não existe (ou e-mail que não confere no guestOrder).
`ORDER_STATUS_INVALID`	A ação não vale para a situação atual do pedido.
`REASON_REQUIRED`	Cancelamento com motivo OTHER sem comentário.
`REFUND_NOTE_REQUIRED`	Cancelar pedido pago sem estorno nem nota de devolução.
`PAYMENT_FAILED`	O Mercado Pago recusou a cobrança (veja statusDetail).
`PAYMENT_APPROVED`	A cobrança acabou de ser paga; o pedido foi confirmado.

10. Referência das consultas e mutações

Todas as entradas da API desta versão (35 consultas e 58 mutações), com quem pode chamar e o início da descrição do `schema.graphql`, que tem os argumentos e os tipos completos. "admin: X" = token do admin com a permissão X no perfil (o perfil Administradores tem todas). "público" = sem login, mas com a chave do aplicativo quando ela é exigida.

10.1 Consultas (query)

Campo	Quem pode	Descrição (do schema)
`store`	público	Uma loja (store view) pelo id.
`stores`	público	Todas as lojas (store views).
`websites`	público	As lojas, na ordem do admin (o cadastro do app oferece a escolha).
`storeInfo`	público	Dados da loja.
`product`	público	Por id ou por SKU (sem diferenciar maiúsculas); informe um dos dois.
`categories`	público	As categorias principais; as outras vêm em children.
`category`	público	Por id ou por url_key; informe um dos dois.
`route`	público	O que abre num endereço da loja.
`products`	público	Lista de produtos com busca, filtros por atributo, categoria e preço, página e as agregações para os filtros.
`mobileHome`	público	A tela inicial do app.
`mobileCategoryProducts`	público	O "mostrar mais" de uma seção de mobileHome.
`mobileOrder`	admin: Iter_Catalog::mobile	Catálogo → Mobile (admin).
`inventorySources`	admin: Iter_Inventory::source	As origens de estoque.
`inventorySource`	admin: Iter_Inventory::source	Uma origem de estoque pelo código.
`customer`	admin: Iter_Customer::customer	Por id ou por e-mail; informe um dos dois.
`me`	cliente (token)	O cliente do token; null sem login de cliente.
`customers`	admin: Iter_Customer::customer	Lista de clientes com busca, tipo (PF/PJ), grupo, website, situação e página.

`attributes`	público (os ocultos só para o admin)	Os atributos de uma entidade (product, customer, customer_address, category), na ordem de posição.
`cart`	público (cartId) ou cliente	O carrinho: do visitante pelo cartId, do cliente logado sem cartId; itens, totais, entrega, pagamento e formas disponíveis.
`orders`	admin: Iter_Sales::order	Lista de pedidos com filtros, ordenação, página e a soma (amount).
`order`	admin: Iter_Sales::order	Pelo id ou pelo número (SP-000001).
`myOrders`	cliente (token)	Os pedidos do cliente do token, o mais novo primeiro.
`myOrder`	cliente (token)	Um pedido do cliente do token pelo número; null se não for dele.
`guestOrder`	público (número + e-mail)	Consulta sem login.
`salesSummary`	admin: Iter_Sales::order	Quantos pedidos e quanto em cada situação, no período (todas as situações vêm, mesmo com 0).
`shippingTableRates`	admin: Iter_Shipping::rates	As faixas de frete do website.
`shippingQuotes`	público	Todas as opções de frete disponíveis para o carrinho e CEP informados, usando o mesmo serviço central do checkout.
`mercadopagoSettings`	público	A Public Key e o máximo de parcelas, para o SDK do Mercado Pago no app.
`customerMessages`	cliente (token)	A conversa do cliente do token com a loja, a mais antiga primeiro.
`customerUnreadMessages`	cliente (token)	Mensagens da loja que o cliente do token ainda não leu.
`adminMessageInbox`	admin: Iter_Message::manage	Um cliente por linha, a conversa mais recente primeiro; 20 por página.
`adminCustomerMessages`	admin: Iter_Message::manage	A conversa de um cliente, a mais antiga primeiro.
`reports`	admin: Iter_Report::*	Os relatórios que o admin do token pode abrir, com os filtros de cada um.
`report`	admin: a permissão do relatório	Um relatório, como na tela.
`currentAdmin`	admin (qualquer)	O usuário do token (ou da sessão do painel); null sem login.

10.2 Mutações (mutation)

Campo	Quem pode	Descrição (do schema)

`createStore`	admin: Iter_Store::store	Cria uma loja (store view).
`updateStore`	admin: Iter_Store::store	Altera uma loja.
`deleteStore`	admin: Iter_Store::store	Exclui uma loja.
`saveMobileOrder`	admin: Iter_Catalog::mobile	Catálogo → Mobile (admin), como o "Salvar" da tela.
`createProduct`	admin: Iter_Catalog::product	Cria um produto simples ou com grade, com os atributos, websites e categorias.
`updateProduct`	admin: Iter_Catalog::product	Só muda o que for enviado; atributos omitidos mantêm o valor.
`saveVariants`	admin: Iter_Catalog::product	A lista enviada é a lista final.
`deleteProduct`	admin: Iter_Catalog::product	Exclui o produto (e as variações).
`uploadProductImage`	admin: Iter_Catalog::product	Envia a imagem de um atributo de imagem do conjunto do produto (troca a atual), pelo multipart request spec com token.
`createCategory`	admin: Iter_Catalog::category	Sem parentId, uma categoria principal do website.
`updateCategory`	admin: Iter_Catalog::category	Só muda o que for enviado.
`moveCategory`	admin: Iter_Catalog::category	Muda a categoria mãe (sem parentId = principal) e a posição.
`setCategoryBanners`	admin: Iter_Catalog::category	Os banners da categoria (como o bloco "Banners" do formulário).
`deleteCategory`	admin: Iter_Catalog::category	Recusada com subcategorias (CATEGORY_HAS_CHILDREN) ou produtos (CATEGORY_HAS_PRODUCTS).
`setProductStock`	admin: Iter_Inventory::stock	Só muda as origens enviadas; as outras mantêm o que têm.
`registerCustomer`	público	Cadastro pela loja.
`confirmCustomerEmail`	público	O token do link do e-mail de confirmação.
`generateCustomerToken`	público	Login do cliente pela loja.
`refreshCustomerToken`	cliente (token)	Troca o token enviado por um novo, com a validade renovada (app mobile).
`revokeCustomerToken`	cliente (token)	Logout.
`requestCustomerPasswordReset`	público	Senha esquecida.
`resetCustomerPassword`	público	O token do link do e-mail + a nova senha.

`updateMe`	cliente (token)	O cliente do token muda os próprios dados; só muda o que for enviado.
`changeMyPassword`	cliente (token)	Troca a senha do cliente do token.
`createMyAddress`	cliente (token)	Novo endereço do cliente do token.
`updateMyAddress`	cliente (token)	Troca o endereço inteiro (o id vem em address.id).
`setMyDefaultAddress`	cliente (token)	Torna o endereço o padrão de entrega e/ou de cobrança.
`deleteMyAddress`	cliente (token)	Remove o endereço.
`createCustomer`	admin: Iter_Customer::customer	Cria um cliente pelo admin, com endereços.
`updateCustomer`	admin: Iter_Customer::customer	Só muda o que for enviado; atributos omitidos mantêm o valor.
`deleteCustomer`	admin: Iter_Customer::customer	Apaga também os endereços e os atributos.
`createCart`	público	Novo carrinho de visitante; devolve o cartId (guarde-o.
`addToCart`	público ou cliente	Soma à quantidade que já estiver no carrinho.
`updateCartItem`	público ou cliente	A nova quantidade do item; 0 remove.
`removeCartItem`	público ou cliente	Tira um item do carrinho.
`mergeCarts`	cliente (token)	Depois do login.
`setCartGuestInfo`	público	Dados do visitante para o pedido.
`setCartShippingAddress`	público ou cliente	O endereço digitado (address) ou um salvo do cliente (customerAddressId).
`setCartShippingMethod`	público ou cliente	Um code de availableShippingMethods.
`setCartPaymentMethod`	público ou cliente	Um code de availablePaymentMethods.
`applyCartCoupon`	público ou cliente	Aplica um cupom no carrinho e devolve os valores recalculados.
`removeCartCoupon`	público ou cliente	Remove o cupom do carrinho e recalcula promoções automáticas.
`placeOrder`	público ou cliente	Fecha o pedido.
`setCartBillingAddress`	público ou cliente	Como o de entrega; sem address nem customerAddressId, a cobrança é no endereço de entrega.
`changeMyOrderPaymentMethod`	cliente (token)	O cliente do token troca a forma de pagamento de um pedido dele que aguarda pagamento (ex..

`confirmOrderPayment`	admin: Iter_Sales::order_payment	Aguardando pagamento → Pagamento confirmado.
`shipOrder`	admin: Iter_Sales::order_ship	Pagamento confirmado → Enviado.
`completeOrder`	admin: Iter_Sales::order_ship	Enviado → Concluído (o cliente recebeu).
`cancelOrder`	admin: Iter_Sales::order_cancel	Antes do envio.
`addOrderComment`	admin: Iter_Sales::order	Comentário no histórico, em qualquer situação.
`createAdminCart`	admin: Iter_Adminsales::create	Um pedido em montagem pelo admin; devolve o cartId, usado nas mutations do carrinho e no placeAdminOrder.
`placeAdminOrder`	admin: Iter_Adminsales::create	Fecha o pedido em montagem.
`saveShippingTableRates`	admin: Iter_Shipping::rates	A lista enviada é a tabela inteira do website (faixa ausente é removida).
`payOrder`	cliente, ou visitante com número + e-mail	Cobra pelo Mercado Pago um pedido "Aguardando pagamento".
`sendCustomerMessage`	cliente (token)	O cliente do token escreve para a loja (1 a 2000 caracteres).
`adminSendCustomerMessage`	admin: Iter_Message::manage	A loja escreve para o cliente, mesmo sem conversa anterior.
`generateAdminToken`	público	Login do admin pela API.
`revokeAdminToken`	admin (qualquer)	Logout.



Iter Store

Projetado e desenvolvido por **Esmerio Neto**

EGSN · esmerioneto@gmail.com

© 2026 EGSN. Todos os direitos reservados.